TECH CHALLENGE

Kalil Gadben de Souza - **Pós Tech - 7IADT - Grupo 36**

Vídeo apresentação: https://youtu.be/z8eQnqCbfFg

Repositório: https://github.com/kalilgadben/fiap-fase3-assistente.git

DESAFIO

Após o sucesso na automação de análises de exames e textos clínicos, o hospital quer avançar para um nível superior de personalização: criar um **assistente virtual médico, treinado com os dados próprios do hospital**, capaz de auxiliar nas condutas clínicas, responder dúvidas de médicos e sugerir procedimentos com base nos protocolos internos.

Além disso, a ideia é organizar fluxos de decisão automatizados e seguros, onde, por exemplo, ao receber informações sobre um paciente, o sistema possa acionar diferentes etapas, como verificar exames pendentes, sugerir tratamentos e emitir alertas para a equipe médica — tudo isso coordenado com **LangChain**.

1. Introdução

Este relatório descreve o desenvolvimento de um assistente virtual médico para o Tech Challenge da Fase 3. Um hospital fictício deseja criar um assistente treinado com dados próprios, capaz de auxiliar médicos em condutas clínicas, responder dúvidas e sugerir procedimentos com base em protocolos internos — tudo coordenado por fluxos automatizados e seguros.

O sistema integra quatro componentes principais:

1. **Fine-tuning de LLM** (GPT-2) com dados médicos públicos do PubMedQA
2. **Pipeline LangChain** para orquestração do modelo com dados do paciente
3. **Fluxo LangGraph** para coordenação automatizada de etapas de decisão
4. **Camada de segurança** com validação, logging de auditoria e explainability

Tecnologia	Finalidade
GPT-2 Small (117M params)	Modelo base para fine-tuning
HuggingFace Transformers	Treinamento e inferência
PubMedQA (pqa_labeled)	Dataset de Q&A clínicas
LangChain	Orquestração do pipeline LLM
LangGraph	Fluxo de decisão automatizado
SQLite	Banco de dados de prontuários

2. Processo de Fine-Tuning

2.1 Escolha do Modelo: GPT-2 Small

Fine-tuning é o processo de ajustar um modelo pré-treinado com dados de um domínio específico, aproveitando o conhecimento linguístico já adquirido. Escolhemos o **GPT-2 Small** (117M parâmetros) por ser leve, executável em CPU, open-source e com amplo suporte no ecossistema HuggingFace. Ele gera texto de forma autorregressiva, prevendo o próximo token com base nos anteriores.

2.2 Dataset: PubMedQA

O **PubMedQA** é um dataset de perguntas e respostas clínicas construído a partir de publicações do PubMed. Utilizamos a versão `pqa_labeled`, com respostas verificadas por especialistas. Cada exemplo contém:

- **Pergunta clínica** baseada em evidências médicas
- **Contexto** com trecho do artigo científico relevante
- **Resposta longa** com explicação detalhada
- **Decisão final** (yes/no/maybe) verificada por experts

2.3 Pipeline de Preprocessing

O preprocessamento foi realizado em quatro etapas:

Etapa 1 — Download

Dados baixados automaticamente via biblioteca `datasets` do HuggingFace, que gerencia download, cache e carregamento de forma transparente.

Etapa 2 — Anonimização

Embora o PubMedQA já seja anonimizado, implementamos função de anonimização via regex para simular o cenário real de um hospital, removendo: padrões de email, números de telefone, datas específicas (DD/MM/AAAA) e nomes próprios com títulos (Dr., Mr., Patient). Em produção, essa etapa seria fundamental para conformidade com a LGPD.

Etapa 3 — Formatação

Cada exemplo foi convertido para formato estruturado com tokens especiais de delimitação:

```
<|inicio|>Pergunta: {pergunta}\nContexto: {contexto}\nResposta:
{resposta}<|fim|>
```

Os tokens `<|inicio|>` e `<|fim|>` foram adicionados ao vocabulário do tokenizador, permitindo que o modelo aprenda a delimitar interações clínicas. O contexto foi limitado a 500 caracteres para evitar sequências excessivamente longas.

Etapa 4 — Divisão Treino/Teste

Dados divididos em **80% treino** e **20% teste** com seed fixa (42) para reprodutibilidade.

2.4 Treinamento

O treinamento utilizou o `Trainer` da HuggingFace, que abstrai o loop de treinamento, cálculo de loss e salvamento de checkpoints. A classe `DatasetMedico` (herdando de `torch.utils.data.Dataset`) carrega e tokeniza os dados. Hiperparâmetros:

Hiperparâmetro	Valor	Justificativa
Épocas	2	Adaptação sem overfitting
Batch size	2	Compatível com CPU (memória RAM)
Learning rate	5e-5	Padrão para fine-tuning
Weight decay	0.01	Regularização leve
Warmup steps	50	Aquecimento gradual do LR
Max length	512 tokens	Limite do GPT-2 Small

O treinamento foi executado com `no_cuda=True`. O modelo final e tokenizador foram salvos em disco para uso pelo assistente.

3. Assistente Médico

3.1 Arquitetura

O assistente foi projetado de forma modular, com quatro módulos independentes:

Módulo	Arquivo	Responsabilidade
Banco de Dados	banco_dados.py	Prontuários, exames e tratamentos (SQLite)
Pipeline LLM	cadeia_llm.py	Integração LangChain + modelo fine-tuned
Fluxo de Decisão	fluxo_grafo.py	Coordenação via LangGraph
Segurança	seguranca.py	Validação, logging e disclaimers

A interface (`aplicacao.py`) oferece terminal interativo onde o médico seleciona pacientes, faz perguntas e visualiza respostas com fontes e disclaimers.

3.2 Banco de Dados

Implementado em **SQLite**, simula o sistema de prontuários do hospital com três tabelas: **pacientes** (10 registros com nome, idade, condição, alergias), **exames** (15 registros com tipo, resultado e status concluído/pendente) e **tratamentos** (10 registros com medicamento, dosagem e observações). A função `montar_contexto_paciente()` agrega todas as informações em texto estruturado que é injetado no prompt do LLM.

3.3 Pipeline LangChain

O pipeline utiliza três componentes do LangChain:

- **HuggingFacePipeline:** carrega o modelo fine-tuned como LLM, com temperature=0.7, top_p=0.9 e repetition_penalty=1.3
- **PromptTemplate:** define regras de comportamento ("nunca prescreva", "sempre sugira") e recebe contexto do paciente + pergunta do médico
- **LLMChain:** conecta template → modelo → resposta, executando o fluxo completo de geração

Como o GPT-2 Small tem limitações na geração livre (tende a gerar em inglês ou com alucinações), implementamos uma **abordagem híbrida**: a função `gerar_resposta_banco()` monta respostas com dados concretos do prontuário. Se o LLM gerar conteúdo útil (filtrado por `limpar_resposta_llm()` que detecta idioma e remove lixo), ele é adicionado como complemento. Caso contrário, só os dados do banco são retornados.

3.4 Fluxo LangGraph

O **LangGraph** permite construir fluxos como grafos dirigidos, onde cada nó processa o estado e passa ao próximo. O estado compartilhado (`EstadoAssistente`) carrega pergunta, contexto, alertas, resposta e fontes entre as etapas.

TRIAGEM ->	CONSULTA ->	GERAÇÃO ->	VALIDAÇÃO
Classifica pergunta por palavras-chave	Busca dados do paciente, gera alertas	LangChain + abordagem híbrida	Segurança, fontes, disclaimer, log

Nó 1 — Triagem

Classifica a pergunta em **exame**, **tratamento** ou **clínica geral** por correspondência de palavras-chave. Essa classificação determina quais alertas gerar e como montar a resposta nos nós seguintes.

Nó 2 — Consulta de Dados

Busca informações do paciente no SQLite. Os alertas são **contextuais**: exames pendentes só aparecem em perguntas sobre exames; alergias só em perguntas sobre tratamento. Isso evita sobrecarga de informação.

Nó 3 — Geração de Resposta

Aciona o pipeline LangChain. A saída do LLM passa pelo filtro de limpeza; se descartada, o fallback (`gerar_resposta_banco`) monta a resposta com dados do prontuário.

Nó 4 — Validação

Verifica prescrição direta, adiciona fontes e disclaimer, e registra a interação no log de auditoria.

4. Segurança e Validação

4.1 Limites de Atuação

Em ambiente hospitalar, um assistente que prescreve sem validação humana pode causar danos graves. O sistema foi projetado com limites rígidos:

- **Nunca prescreve:** toda resposta é formulada como sugestão
- **Disclaimer obrigatório:** toda resposta inclui aviso de necessidade de validação médica
- **Bloqueio ativo:** respostas com termos de prescrição direta são bloqueadas automaticamente

4.2 Detecção de Prescrição

Uma lista de termos proibidos ("prescrevo", "receito", "tome", "administre", "suspenda o uso", "aumente a dose") é verificada antes de cada resposta. Se detectados, a resposta original é substituída por mensagem de bloqueio informando quais termos foram encontrados.

4.3 Logging de Auditoria

Todas as interações são registradas em `logs/auditoria.log` com:

Campo	Descrição	Exemplo
Timestamp	Data e hora	2024-03-20 14:35:22
Pergunta	Texto do médico	Quais exames pendentes?
Resposta	Primeiros 200 caracteres	O paciente possui 1 exame...
Fontes	Origem dos dados	Banco de dados, PubMedQA
Status	Aprovada ou bloqueada	APROVADA

4.4 Explainability

Cada resposta indica suas **fontes**: "Modelo fine-tuned (PubMedQA)" para informações do LLM e "Banco de dados do hospital" para dados do prontuário. Essa transparência permite que o médico avalie a confiabilidade da resposta.

5. Avaliação do Modelo e Análise dos Resultados

5.1 Métrica: Perplexidade

A métrica principal foi a **Perplexidade**, que mede o quão "surpreso" o modelo fica ao processar um texto. Calculada como `PPL = e^(loss média)`, quanto menor a perplexidade, melhor o modelo entende aquele domínio. Se um modelo tem perplexidade 12, significa que está "indeciso" entre ~12 próximos tokens possíveis — muito menos que os ~37 do modelo não ajustado.

5.2 Resultados

Avaliação realizada com 20 exemplos do conjunto de teste:

Modelo	Perplexidade
GPT-2 Base (sem fine-tuning)	37.01
GPT-2 Fine-tuned (PubMedQA)	12.71
Melhoria	**65.67%**

> **✓ Redução de 65.67% na perplexidade — o modelo aprendeu o domínio médico.**

O modelo base (37.01) não possui familiaridade com vocabulário clínico como "ecocardiograma" ou "hemoglobina glicada". Após o fine-tuning (12.71), passou a reconhecer terminologias e padrões de linguagem médica com muito mais confiança. Considerando que usamos apenas o GPT-2 Small em CPU com 2 épocas, o resultado é expressivo.

5.3 Limitações Qualitativas

Apesar da melhoria na perplexidade, observamos limitações na geração de texto livre:

- **Idioma:** o GPT-2 foi pré-treinado em inglês e tende a gerar respostas nesse idioma
- **Alucinações:** respostas gramaticalmente corretas mas clinicamente irrelevantes
- **Repetições:** o modelo por vezes entra em loops gerando a mesma frase

Essas limitações foram contornadas com a **abordagem híbrida** (dados do banco + LLM filtrado) e o módulo `limpar_resposta_llm()` que detecta idioma e descarta conteúdo inadequado.

5.4 Análise Geral

> **Pontos positivos**

- **65.67% de melhoria** na perplexidade com fine-tuning
- Integração funcional LLM + banco de dados + fluxo de decisão
- Abordagem híbrida garante respostas úteis mesmo com LLM limitado

- Sistema de segurança efetivo com bloqueio e auditoria

Melhorias futuras

- **Modelo maior:** LLaMA 2 ou Falcon 7B com QLoRA para respostas mais sofisticadas
- **RAG:** busca em protocolos médicos para respostas fundamentadas em tempo real
- **Interface web:** Streamlit ou Gradio para uso pela equipe médica
- **Dataset PT-BR:** dados médicos em português para melhorar geração no idioma nativo